# How Would You Design AxeDrobe?

Sure. I would design AxeDrobe as a scalable fashion e-commerce and order-management system.

Before directly discussing the architecture, I would first clarify the system requirements, estimate the expected traffic, and then explain the services, APIs, database design and order-processing flow.

## 1. Functional requirements

For the MVP, the system should support two main types of users: customers and administrators.

A customer should be able to:

- Register and log in
- Browse and search for products
- View product information
- Place an order
- View order details and order history
- Receive an order confirmation

An administrator should be able to:

- Add products
- Update products
- Delete products
- Manage product information
- View available stock

Internally, the system should also:

- Validate the user before accepting an order
- Validate the current product price
- Check and update inventory
- Store the order
- Publish an order event
- Send an order notification

For the initial scope, I would keep payment processing, shipping, returns and recommendation systems outside the MVP. They can be added later as separate services.

## 2. Non-functional requirements

The most important non-functional requirements are scalability, availability, security, consistency and fault tolerance.

Product browsing is a read-heavy operation, so product APIs should have low latency and support high traffic.

Order creation requires stronger consistency because we should not create duplicate orders, charge an incorrect price or sell more items than are available.

The system should also be fault-tolerant. For example, if the notification service is temporarily unavailable, the order should still be processed and the notification should be retried later.

Security is also important. Passwords should be hashed, protected APIs should require JWT authentication, and role-based access should prevent customers from accessing administrator operations.

## 3. Scale estimation

To discuss scalability, I would make some assumptions.

Let us assume AxeDrobe has:

- 1 million daily active users
- Around 20 product-related requests per user per day
- Around 100,000 orders per day

That would result in approximately 20 million product requests per day.

The average would be around 230 requests per second, but during peak hours or a sale, the system may receive 2,000 to 3,000 product requests per second.

For orders, 100,000 orders per day is only around 1.2 orders per second on average. However, e-commerce traffic is highly uneven. During flash sales, we may receive hundreds of order requests per second.

If one stored order, including its items, requires approximately 2 KB, then 100,000 orders would generate around 200 MB of order data per day, or roughly 73 GB per year before indexes, backups and replicas.

These numbers show that the product side is read-heavy, while the order and inventory side needs to handle sudden traffic spikes and concurrency.

## 4. High-level architecture

At a high level, the customer uses the React frontend.

The frontend communicates with an API Gateway or load balancer. The gateway becomes the common entry point for authentication, routing, rate limiting and request tracking.

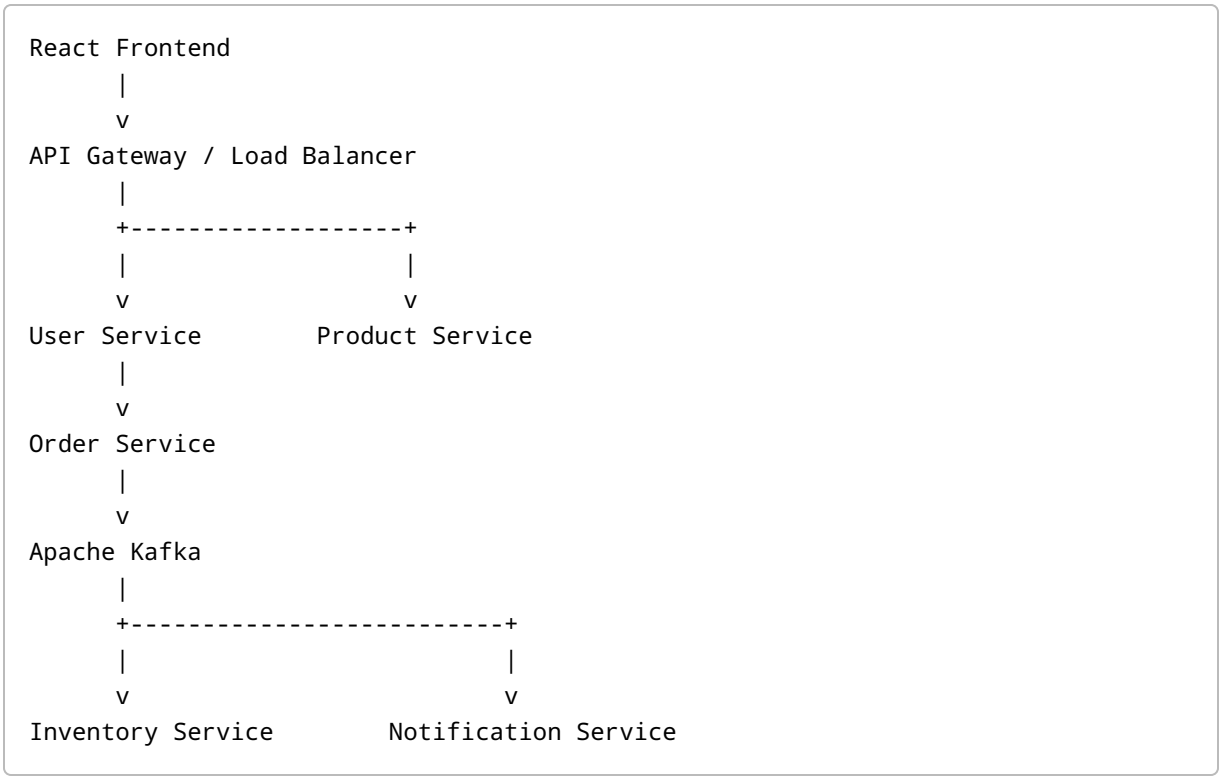
Behind the gateway, I would have the following services:

1. User Service

2. Product Service
3. Order Service
4. Inventory Service
5. Notification Service

Each service has a separate responsibility and should own its own data.

The basic architecture would look like this:




The User Service handles registration, login, JWT generation and roles.

The Product Service handles product creation, updates, deletion, listing and search.

The Order Service handles order creation and order history.

When an order is created, the Order Service publishes an event to Kafka.

The Inventory Service consumes the event and updates the stock.

The Notification Service consumes the event and sends an order confirmation. In the current project, the notification is printed in the service logs. In a production version, this could be replaced with email, SMS or push notifications.

## 5. Why I used microservices

I separated the system into microservices because each part of an e-commerce application has different scaling and reliability requirements.

For example, the Product Service receives many read requests, while the Order Service receives fewer but more critical write requests.

The Notification Service should not slow down order creation. By processing notifications asynchronously, an order does not have to wait for an email or SMS provider.

Microservices also allow individual services to be deployed and scaled separately.

However, I would also explain that microservices add complexity. For a very small application, a modular monolith would be simpler.

In fact, I first designed the project as a monolith to understand Spring Boot, REST APIs and MySQL integration. After that, I separated it into services and introduced Kafka. This helped me understand both approaches rather than using microservices only as a keyword.

## 6. Order creation flow

The most important flow in this project is order creation.

The customer first logs in and receives a JWT.

The customer then browses products and submits an order request.

The frontend should send only the product IDs and quantities. The server should never trust a price sent by the frontend because the customer could modify it.

The Order Service performs the following steps:

1. Validate the JWT and identify the customer.
2. Validate the request.
3. Fetch the latest product information from the Product Service.
4. Calculate the total amount on the backend.
5. Create an order with a `PENDING` status.
6. Save the order and its items in MySQL.
7. Publish an event to Kafka.
8. Return the order ID and current status to the customer.

An example event could look like this:

```
{
  "eventId": "e18c7a2f",
  "eventType": "ORDER_CREATED",
  "orderId": 1001,
  "userId": 1,
  "items": [
    {
      "productId": 10,
      "quantity": 2
    }
  ],
  "createdAt": "2026-07-28T10:30:00Z"
}
```

The Inventory Service consumes this event, checks the available quantity and reduces the stock.

The Notification Service also consumes the event and sends a confirmation.

## 7. Improving the current order flow

In the current MVP, the `order-created` event is used to reduce the inventory after the order is saved.

That works for demonstrating Kafka-based communication, but in a production system, there is a possible problem.

Two customers could place an order for the final available item at almost the same time. Both orders might be created before the inventory is updated. This could result in overselling.

To solve this, I would introduce an inventory reservation flow.

The Order Service would create the order with a `PENDING` status and publish an `INVENTORY_RESERVATION_REQUESTED` event.

The Inventory Service would attempt to reserve the stock and publish one of two events:

```
INVENTORY_RESERVED
```

or:

```
INVENTORY_REJECTED
```

If inventory is reserved, the Order Service changes the order status to `CONFIRMED`.

If inventory is unavailable, the Order Service changes the status to `REJECTED` or `CANCELLED`.

This is a Saga-style approach where each service completes its own local transaction and communicates through events.

## 8. Preventing overselling

Inside the Inventory Service, the stock update must be atomic.

One solution is to execute an SQL query similar to:

```
UPDATE inventory
SET available_quantity = available_quantity - :requestedQuantity
WHERE product_id = :productId
  AND available_quantity >= :requestedQuantity;
```

After executing the query, I would check the number of affected rows.

If one row was updated, the reservation was successful.

If zero rows were updated, there was not enough stock.

This is safer than first reading the stock and then updating it because another request could modify the stock between those two operations.

Another option is optimistic locking using a version column.

## 9. API design

The authentication APIs would be:

```
POST /api/auth/register
POST /api/auth/login
GET  /api/users/profile
```

The product APIs would be:

```
GET    /api/products
GET    /api/products/{productId}
POST   /api/products
```

```
PUT    /api/products/{productId}
DELETE /api/products/{productId}
```

The administrator product operations would require the `ADMIN` role.

The order APIs would be:

```
POST /api/orders
GET  /api/orders/{orderId}
GET  /api/orders
```

An order creation request could be:

```
{
  "items": [
    {
      "productId": 10,
      "quantity": 2
    },
    {
      "productId": 15,
      "quantity": 1
    }
  ]
}
```

The response could be:

```
{
  "orderId": 1001,
  "status": "PENDING",
  "totalAmount": 4997.00
}
```

I would also accept an idempotency key in the order request.

For example:

```
Idempotency-Key: 702bf7c4-22fb-4ecb
```

If the customer double-clicks the order button or the frontend retries because of a network issue, the same key prevents the system from creating two separate orders.

## 10. Database design

I would use MySQL because users, products, orders and inventory are structured and transactional.

Each microservice should own its database or schema.

### User Service

```
users
-----
id
name
email
password_hash
role
created_at
```

The email should have a unique index.

### Product Service

```
products
--------
id
name
description
price
category
image_url
active
created_at
updated_at
```

For faster browsing, I would create indexes on category, name and active status.

### Order Service

```
orders
------
id
user_id
status
total_amount
idempotency_key
```

```
created_at
updated_at
```

```
order_items
-----------
id
order_id
product_id
product_name
unit_price
quantity
subtotal
```

I would save the product name and price inside `order_items` as a snapshot.

This is important because the product price or name may change later, but the old order must still display the information that was valid when the customer placed it.

### Inventory Service

```
inventory
---------
product_id
available_quantity
reserved_quantity
version
updated_at
```

The Inventory Service should be the single source of truth for stock. I would avoid maintaining editable stock values in both the Product Service and Inventory Service because that could create inconsistent data.

## 11. Kafka design

Kafka helps decouple order processing from inventory and notification processing.

The Order Service acts as a producer.

The Inventory Service and Notification Service run in separate consumer groups, so both receive the same event independently.

I would include the following fields in every event:

- Event ID
- Event type

- Event version
- Order ID
- Timestamp
- Payload

The event ID helps detect duplicate messages.

The event version allows the event structure to change safely in the future.

The order ID can be used as the Kafka message key so that events belonging to the same order remain in the correct order within a partition.

## 12. Handling duplicate Kafka events

Kafka generally provides at-least-once delivery, which means a consumer may receive the same event more than once.

Therefore, the Inventory Service must be idempotent.

I would create a table such as:

```
processed_events
----------------
event_id
processed_at
```

Before updating inventory, the consumer checks whether the event ID has already been processed.

If it has already been processed, the consumer ignores it.

If it has not been processed, the inventory update and insertion into `processed_events` should happen inside the same database transaction.

This prevents duplicate stock deductions.

## 13. Handling database and Kafka consistency

There is another important failure case.

Suppose the Order Service saves the order in MySQL, but Kafka becomes unavailable before the event is published.

The order would exist in the database, but the Inventory Service would never receive the event.

To solve this, I would use the transactional outbox pattern.

The Order Service would save both the order and an outbox record inside the same MySQL transaction.

```
outbox_events
-------------
id
aggregate_id
event_type
payload
status
created_at
```

A background publisher would continuously read unpublished outbox records and send them to Kafka.

After successful publishing, it would mark the event as published.

This ensures that an order event is not silently lost.

## 14. Failure handling

If the Product Service is unavailable, product browsing may use cached data, but order creation should not use an old price without validation.

If the Inventory Service is temporarily unavailable, the order remains in `PENDING` status and the event can be retried.

If a Kafka consumer repeatedly fails, the message should be sent to a dead-letter topic for investigation.

If the Notification Service fails, the order should not be cancelled. Notification is a secondary operation, so it should be retried independently.

I would use:

- Retry with exponential backoff
- Dead-letter topics
- Timeouts
- Circuit breakers
- Idempotent consumers
- Correlation IDs

## 15. Security

The User Service handles registration and login.

Passwords should be stored using BCrypt or another secure password-hashing algorithm.

After successful login, the service generates a JWT containing the user ID and role.

The frontend sends the token in the request:

```
Authorization: Bearer <token>
```

The system supports two roles:

```
ADMIN
CUSTOMER
```

Customers can browse products and manage their own orders.

Administrators can create, update and delete products.

I would validate authentication at the gateway and again inside sensitive services. This prevents a service from depending only on the gateway for security.

I would also add:

- Request validation
- Rate limiting
- Secure secret management
- Short-lived access tokens
- Refresh tokens
- Audit logs for administrator operations

## 16. Scaling the system

All Spring Boot services should remain stateless so that multiple instances can run behind a load balancer.

The Product Service can be scaled horizontally because it receives the largest amount of traffic.

Frequently accessed product data can be cached in Redis.

Product images and frontend files can be served through object storage and a CDN instead of through the Product Service.

Kafka consumer groups allow multiple Inventory Service or Notification Service instances to process different partitions in parallel.

For MySQL, I would use:

- Proper indexes

- Connection pooling
- Read replicas for read-heavy operations
- Database backups
- Partitioning or archival for old orders when the dataset becomes very large

## 17. Monitoring and observability

In a microservices system, a single request may travel through several services.

I would assign a correlation ID to every request and include it in logs and Kafka events.

For example, when investigating order `1001`, I should be able to trace:

```
Frontend
   -> API Gateway
   -> Order Service
   -> Kafka
   -> Inventory Service
   -> Notification Service
```

I would monitor:

- API response time
- Error rate
- Order creation success rate
- Pending orders
- Kafka consumer lag
- Inventory reservation failures
- Database connection usage
- Dead-letter topic size

For implementation, tools such as Spring Boot Actuator, Micrometer, Prometheus, Grafana and distributed tracing could be added.

## 18. Why I selected these technologies

I used Java 17 and Spring Boot because they provide a strong ecosystem for REST APIs, security, database access and Kafka integration.

I used Spring Data JPA and Hibernate to simplify database operations.

I selected MySQL because orders and inventory require structured relational data and transactions.

I used Spring Security and JWT for stateless authentication and role-based access.

I used Kafka because inventory and notification processing should be decoupled from the main order request.

I used Swagger/OpenAPI to document and test the APIs.

I used Docker Compose so the frontend, backend services, MySQL, Kafka and ZooKeeper could be started in a consistent local environment.

## 19. Current implementation versus production version

In my current AxeDrobe implementation, I have:

- A React frontend
- User Service
- Product Service
- Order Service
- Inventory Service
- Notification Service
- MySQL integration
- JWT authentication
- Role-based access
- Kafka-based order events
- Swagger API documentation
- Docker Compose configuration

The Order Service publishes the `order-created` event.

The Inventory Service consumes it and updates stock.

The Notification Service consumes it and currently logs an order notification.

For a production version, my next improvements would be:

- API Gateway
- Payment Service
- Inventory reservation
- Saga-based order status handling
- Transactional outbox
- Idempotent Kafka consumers
- Redis caching
- Retry and dead-letter topics
- Email or SMS notifications
- Monitoring and distributed tracing
- Cloud deployment and automated CI/CD

## Conclusion

The main idea behind AxeDrobe is to keep each business responsibility separate.

The User Service handles identity.

The Product Service handles the catalogue.

The Order Service manages the order lifecycle.

The Inventory Service owns stock.

The Notification Service handles customer communication.

Synchronous APIs are used when an immediate answer is required, such as login, product lookup and order creation.

Kafka is used for operations that can be processed asynchronously, such as inventory events and notifications.

This design keeps the services independent and scalable, while techniques such as inventory reservation, idempotency, the outbox pattern and Saga-based processing help maintain correctness when failures occur.